

Déséquilibre des classes

Quand une classe est rare (fraude, défaut de crédit)... · Marianne A. · compagnon de M1

Le problème. Quand une classe est **rare**, le modèle « par défaut » l'ignore et l'**accuracy trompe**. Avec 95 % de négatifs, un modèle qui prédit *tout négatif* obtient 95 % d'accuracy... et **0 détection** de la classe rare. Le score paraît excellent, le modèle est inutile.

La parade. Quatre leviers complémentaires : la **bonne métrique**, la **pondération**, le **rééchantillonnage**, et le **seuil de décision**.

Rappel Ici on juge sur le **RAPPEL** de la classe rare et le **F1 macro**, pas sur l'accuracy.

1 D'abord, la bonne métrique

MÉTRIQUE	POURQUOI ICI
Accuracy	À proscrire seule : trompeuse en déséquilibre (95 % de négatifs → 95 % d'accuracy en prédisant tout négatif).
Rappel (recall) de la minoritaire	Combien de cas rares on retrouve — souvent LA cible (attraper les fraudes, les défauts).
Précision de la minoritaire	Combien d'alertes sont justes — mesure le coût des fausses alertes.
F1 macro	Moyenne des F1 par classe à poids égal : la rare compte autant que la fréquente.
PR-AUC (précision-rappel)	Plus informative que ROC-AUC en fort déséquilibre ; se concentre sur la classe rare.

Précision ou rappel ? Le choix dépend du **coût des faux positifs vs faux négatifs** : rater une fraude (faux négatif) coûte cher → on privilégie le rappel ; multiplier les fausses alertes coûte cher → on privilégie la précision.

2 Les leviers d'action

LEVIER	EN QUOI	À SAVOIR
class_weight="balanced"	Pondère la perte pour compenser la rareté de la minoritaire.	Simple, à ESSAYER EN PREMIER (LogReg, RandomForest, SVM ; HistGB via sample_weight).

LEVIER	EN QUOI	À SAVOIR
Oversampling (SMOTE)	Génère des exemples synthétiques de la classe minoritaire.	Sur le TRAIN uniquement , via un pipeline imblearn.
Undersampling	Réduit la classe majoritaire pour rééquilibrer.	Simple, mais perd de l'information (exemples jetés).
Ajuster le seuil de décision	Ne pas rester à 0.5 : déplacer le seuil de probabilité.	Choisir via la courbe précision-rappel selon le compromis voulu.

3 Le faire proprement

Le rééchantillonnage se fait **UNIQUEMENT sur le train**, **JAMAIS** sur la validation ni le test, et **DANS le pipeline** pour rester correct en validation croisée : à chaque fold, SMOTE ne voit que la partie entraînement du fold, jamais la partie validée.

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

pipe = ImbPipeline([
    ("pre", pre),
    ("smote", SMOTE(random_state=42)), # ne s'applique qu'au TRAIN de chaque fold
    ("clf", clf),
])

# StratifiedKFold pour la CV, scoring="f1_macro"
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cross_val_score(pipe, X_train, y_train, cv=cv, scoring="f1_macro")
```

Alternative sans resampling. Souvent `class_weight="balanced"` (ou `sample_weight`) suffit : pas d'exemples synthétiques à générer, un seul paramètre à activer — c'est le premier réflexe avant SMOTE.

4 Pièges

LE PIÈGE	LE CORRECTIF
SMOTE / rééchantillonnage avant le split	Fuite de données → le faire dans le pipeline , sur le train seulement.
Optimiser l'accuracy	Verdict faux en déséquilibre → viser F1 macro + rappel de la minoritaire.

LE PIÈGE	LE CORRECTIF
Rééchantillonner le test / la validation	Scores irréalistes → ne jamais toucher au test ni à la validation.
Oublier stratify	Folds non représentatifs → StratifiedKFold + stratify=y au split.
Laisser le seuil à 0.5	Rappel trop bas → l' ajuster via la courbe précision-rappel.

La règle d'or. Bonne métrique **d'abord** (F1 macro, rappel de la classe rare) · `class_weight` en **premier réflexe** · rééchantillonnage seulement sur le **train**, dans le pipeline · ajuste le **seuil** selon le coût métier des erreurs (faux positifs vs faux négatifs).

À retenir. En déséquilibre, l'accuracy ment : juge sur le rappel de la classe rare et le F1 macro, pondère (`class_weight`) avant de rééchantillonner, ne touche au rééchantillonnage que sur **Fiche 10/13 · Série ML** le train dans le pipeline, et règle le seuil selon le coût des erreurs.